

3 주차 진행 공유

시험 벼락치기 스케줄러

시험기간 대학생들을 위한 수면 · 카페인 스케줄 최적화 도구 — 다중 시험 최적화 · 화면 연동 파트

N110_ 안정연

무엇을, 왜 만드는가

상황

시험기간이 되면 대학생 대부분은 벼락치기를 하고, 수면 · 카페인 섭취를 순전히 감에 의존해 조절한다. 그 결과, 시험이 여럿 겹치는 주간 전체를 고려하지 못하고 수면부족 · 컨디션 저하가 누적된다.

만드는 것

시험 날짜 · 공부량 · 수면패턴 · 카페인 상태를 입력하면, 검증된 생물수학 모델로 " 시험 시작 시각에 각성도가 최고가 되는 " 수면 · 카페인 스케줄을 계산해주는 도구.

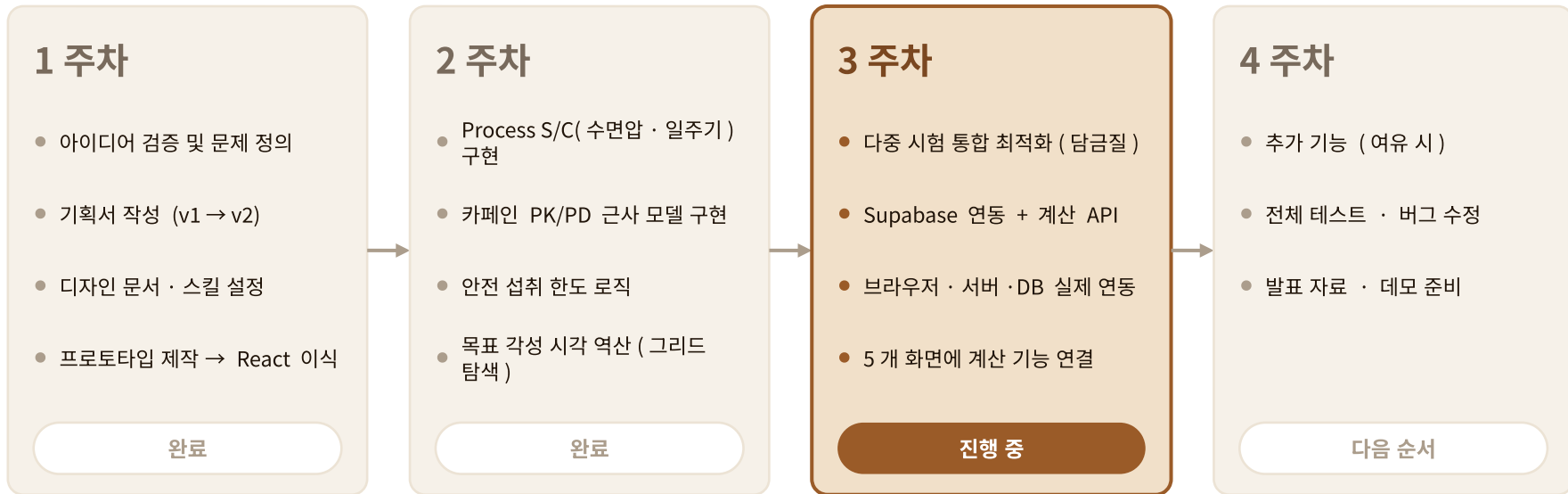
핵심 기능

하루 단위 스케줄이 아니라, 시험이 여러 개 겹치는 " 시험기간 전체 " 를 한 번에 고려해 수면 · 카페인 스케줄을 배분해준다.

사용한 모델

Two-Process Model(수면압 + 일주기리듬) + 카페인 약동학 · 약력학 (PK/PD) 상호작용 모델
— 수면과학 논문 기반의 검증된 공식 사용

프로젝트 로드맵 (4 주)



지금 여기: 3 주차 — "계산하기"를 누르면 실제로 계산돼 결과가 나오는, 끝까지 동작하는 앱을 완성하는 게 목표

3 주차 목표 · 진행 상황 (07/20 ~ 07/23)

목표 : 흩어져 있던 계산 엔진 · 화면 · DB 를 하나로 연결해
— " 계산하기 " 를 누르면 여러 시험을 한 번에 고려한 스케줄이 실제로 나오게 만들기

월 07/20

계산 엔진 코어

- 다중 시험 최적화 3 종 구현
- 후보 그리드 · 목적함수 · 국소 탐색
- + 최소 수면시간 패널티

화 07/21

검증 · 버그 · DB

- 검증 스크립트로 수렴 확인
- 기상 시각 · 축 분리 버그 수정
- Supabase 프로젝트 · 마이그레이션

수 07/22

연동 기반 · 화면

- POST
/api/schedule/calculate
- 프록시 · Context 연동 기반
- 입력 · 처리중 · 결과 화면 API 연결

목 07/23

결과 마무리

- 각성도 그래프 렌더 · 가로 스크롤
- localStorage 최근 기록
- 스케줄 조정 재계산 연동

3 주차 · 완료 이슈

3 주차에 달은 이슈 (GitHub)

다중 시험 최적화부터 화면 연동까지 — 3 주차 마일스톤 이슈 18 개를 모두 닫았다



계산 엔진 (BE)

- #10 다중 시험 그리드 결정변수 정의
- #11 목적함수 (최솟값 채점) 구현
- #12 로컬 탐색 휴리스틱 (담금질)
- #13 다중 시험 검증 스크립트
- #21 최소 수면시간 패널티 반영
- #22 추천 기상 시각 버그 수정
- #23 취침 · 기상 축 분리



연동 · DB (BE/DB)

- #14 Supabase 생성 · 마이그레이션
- #15 POST /schedule/calculate
- #24 연동 기반 (프록시 +Context)



화면 (FE)

- #16 정보 입력 화면 API 연동
- #17 처리 중 실제 응답 대기
- #18 결과 추천 · 경고 렌더링
- #25 각성도 그래프 렌더링
- #26 각성도 그래프 가로 스크롤
- #19 localStorage 최근 기록
- #20 스케줄 조정 재계산 연동
- #29 기본 시험 시드 버그 수정

기술 스택

서비스 기술 지도



FE — React

5 개 화면이 mock 없이 실제 계산 API 와 연결됨 .
각성도 그래프 · 추천 스케줄 · 최근 기록까지 동작



BE — Express

POST /api/schedule/calculate 로 다중 시험
최적화 엔진을 노출 , Supabase 참고데이터 조회



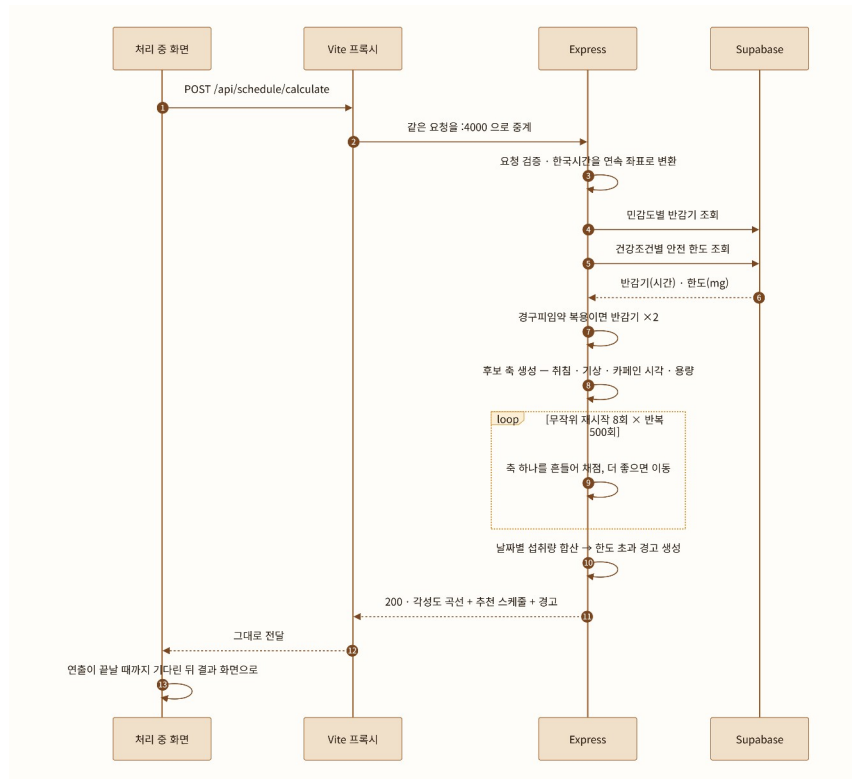
DB — Supabase
(Postgres)

민감도별 반감기 · 안전 섭취 한도 등 참고
데이터를 실제 SELECT 로 조회 — 연결 완료

핵심 기능 하나 : 계산하기 (입력 → 계산 엔진 → 결과)

정보 입력 화면 " 계산하기 " → POST /api/schedule/calculate → 입력 검증 → DB 에서 반감기 · 안전 한도 조회 → Two-
Process+ 카페인으로 각성도 곡선 → 다중 시험 로컬 탐색 → 안전 한도 판정 → { 각성도 곡선 , 추천 스케줄 , 경고 } 응답

서버 처리 흐름 — POST /api/schedule/calculate



어떻게 최적 스케줄을 찾을까 — 계산 3 단계



1 단계

후보 만들기

`multiDayCandidates.ts`

취침 · 기상 · 카페인 시각 · 용량을
각각 "축"으로 둔다. 평소값 ± 1 시간을
15 분 간격으로 → 밤마다 최대 9 개 후보.
밤 4 개면 조합이 81^4 이라, 전체 조합은
안 만들고 축 목록만 반환한다.



2 단계

점수 매기기

`multiDayObjective.ts`

스케줄 하나 = 시험별 시작 시각의
각성도 중 최솟값 (min).
"가장 컨디션 나쁜 시험"을 끌어올리는
방향으로 채점하고, 여기서 패널티
(수면 부족 · 수면 중 섭취 · 한도 초과)를 뺀다.



3 단계

좋은 쪽으로 옮기기

`multiDayLocalSearch.ts`

담금질 (simulated annealing). 축 하나를
무작위로 골라 ± 1 칸 옮겨 채점 → 좋으면
이동, 나빠도 초반엔 확률적으로 허용.
무작위 재시작 8 회 × 반복 500 회 중
가장 높은 점수를 최종 채택한다.

→ 물리적으로 말이 안 되는 후보 (기상 전 카페인, 시험 후 기상)는 1 단계에서 이미 제외 — 자세한 채점 · 탐색은 다음 장

점수는 어떻게 매길까 — 가산점 · 패널티와 국소 탐색

채점 공식

점수 = 시험별 각성도의 최솟값 - 패널티 3 종

패널티 3 종 (점수에서 뺀다)

최소 수면시간 부족

부족 1 시간당 - 0.5

자는 중 카페인 섭취

기상까지 남은 1 시간당 - 1.0

하루 안전 한도 초과

100mg 초과당 - 0.5 (0.005/mg)

가산점은 따로 없다 — " 시험 시각에 각성도가 높다 " 가 곧 점수 . 안전 관련 패널티는 각성도 차이 (보통 0.01~0.05) 보다 세게 잡아 항상 이기게 했다 .

국소 탐색 — 담금질 휴리스틱

왜 전수 탐색을 안 하나 — 밤이 몇 개만 돼도 후보 조합이 81^n 으로 폭발해 다 못 뒤진다 .

- ① 축 하나를 무작위로 골라 ± 1 칸 옮긴다 (이웃)
- ② 채점해서 더 좋으면 그 이웃으로 이동
- ③ 나쁘더라도 " 온도 " 에 따른 확률로 가끔 이동
→ 국소 최적점에 갇히지 않게
- ④ 온도는 반복마다 기하급수로 낮아져 , 후반엔 사실상 오르막으로만 이동 (수렴)
- ⑤ 무작위 초기값으로 8 번 재시작 → 최고 채택

탐색 파라미터

500

반복

8

재시작

0.1

초기온도

0.001

종료온도

데모

브라우저 · 서버 · DB 를 연결한 실제 동작 화면

DEMO



<http://localhost:5173/>

"계산하기"를 누르면 앞 장의 시퀀스 (POST /api/schedule/calculate) 가 실제로 실행됩니다 — 발표 중 다른 창에서 직접 시연

AI 와 어떻게 일했나 — 3 주간의 작업 방식

작업 순서 — 이슈 하나를 처리하는 사이클



쓰는 Skill 3 개 (.claude/skills/)

/commit

" 커밋 " 한마디로 바로 실행 — Conventional Commits
한국어 형식, 파일 하나당 커밋 하나로 분리

/new-component

새 컴포넌트 요청 시 CLAUDE.md · 디자인 .md 를 먼저 읽고
디자인 토큰 재사용, 문서에 없으면 임의로 안 정하고 질문

/daily-recap

하루 마무리 회고 — 한 일 · 목적 · 세부사항 ·
햇갈린 점 · 배운 것 5 가지를 표로

Agent · 함께 정한 규칙

Agent — 기본은 메인 세션에서 직접 진행

- Explore — 여러 파일을 한 번에 훑어야 할 때 코드베이스 탐색
- Plan — 구현 전에 단계별 계획을 먼저 받아볼 때

함께 정한 규칙 — CLAUDE.md 에 적어둔 것

- 지시하지 않은 작업은 AI 가 스스로 실행하지 않기
- 빈틈은 임의로 채우지 말고 다시 질문하기
- 코드 설명은 목적 → 방식 → 근거 → 구현 4 단계로

다음 주 할 일 · 고민되는 점



4 주차 할 일

- 추가 기능 (시간 여유 있으면)
- 전체 테스트 · 버그 수정
- 발표 자료 · 데모 준비
- 남은 선택 이슈 정리

· #27 홈 화면 시험 캘린더

· #28 FE·BE 타입 통합

고민되는 점

1 FE·BE 타입을 통합할까 (#28)

client(React) 와 server(Express) 가 요청 · 응답 타입을 각자 정의 중 . 공유 폴더로 합쳐 한 곳에서 관리할지 , 계층을 나눠 그대로 둘지 고민

2 결과 화면을 어떻게 마무리할까

각성도 그래프 · 추천 스케줄 · 경고 · 캘린더를 한 화면에 어디까지 담고 어떻게 정리해야 사용자가 한눈에 이해할지

3 결과 캘린더의 정보 범위

캘린더에 시험 · 수면 · 카페인 · 공부 중 어디까지 통합해 보여줄지 — 너무 많으면 복잡 , 적으면 밋밋